

AI 评价生成 Demo 最终交付文档

1. 项目概述

本项目实现了一个可公开访问的移动端 H5 Demo，用于帮助 Sunny Tea House 顾客快速生成平台评价或推荐文案。用户选择 1-2 个消费感受，再选择 Google 或小红书，前端请求 FastAPI 后端调用真实大模型生成内容。用户可以继续手动编辑正文，然后一键复制，并在页面内查看企业微信 Webhook 的 mock 请求拼装结果，最后跳转到对应平台入口。

当前最终交付模式为：

- 前端：Vue 3 + Vite 移动端 H5。
- 后端：Python FastAPI，部署在 Render。
- 模型：后端读取环境变量调用真实 OpenAI-compatible 模型接口。
- 企业微信：不接入真实企业微信群机器人，改为纯前端 mock 演示 Webhook 数据拼装与调用逻辑。
- 数据库：本地 Docker PostgreSQL 已预留 reviews 表和 /api/incoming-review 接口；当前主流程不依赖数据库。

线上地址：

前端地址：<https://28344e2d.pinme.dev>

2. AI 辅助编程工具使用策略

本项目使用 Codex 参与需求阅读、方案拆解、代码实现、调试验证、部署辅助和交付文档整理。AI 工具主要承担高重复度、结构化和跨文件联动工作，关键技术路线和范围调整由人工确认。

AI 参与内容：

- 从需求文档中提取核心闭环：选择感受、选择平台、生成评价、用户编辑、复制、跳转平台、Webhook 演示。
- 规划前后端分离结构，前端使用 Vue/Vite，后端使用 FastAPI。
- 编写后端单元测试，覆盖平台校验、感受数量限制、Prompt 选择、空模型返回重试、Webhook 预留逻辑和 PostgreSQL 预留仓库。
- 搭建移动端页面，补充 loading、失败提示、复制失败提示、后退后可继续操作、生成后只能手动修改等状态。
- 协助 Render、PinMe 部署与 CORS 排查。
- 整理启动命令、依赖导入路径、部署说明、未完成项和本交付文档。

人工确认内容：

- 真实模型可以接入。
- 真实企业微信不接入，改为纯前端 mock 展示 Webhook 拼装逻辑。
- 后端部署到 Render，前端部署到 PinMe。
- 模型返回空内容的根因是生成token 上限过小，最终将生成接口 max_tokens 调整为 20000。

3. 功能实现说明

3.1 前端功能

前端主页面实现了完整移动端流程：

- 消费感受标签：最多选择 2 个，至少选择 1 个才允许生成。
- 平台切换：Google 和小红书两种模式。
- 生成内容：调用后端 /api/generate-review，生成成功后锁定生成按钮，后续只能手动编辑。
- 可编辑正文：用户可以直接修改 AI生成内容，最终发布内容不限制长度。
- 复制并模拟 Webhook：复制正文后，在前端构造企业微信机器人 Markdown 请求体，并展示请求方式、模拟地址、中文摘要、回复草稿和 JSON body。
- 打开发布入口：Google 打开Google Maps/Search 入口，小红书打开网页入口，不做内容预填。
- 页面后退恢复：从平台入口后退回评价页面后，仍可再次点击“打开发布入口”和“复制并模拟 Webhook”。

3.2 后端接口

后端提供以下接口：

```
GET /api/health
POST /api/generate-review
POST /api/notify-wecom
POST /api/incoming-review
```

当前主流程实际使用：

```
GET /api/health
POST /api/generate-review
```

/api/notify-wecom 和 /api/incoming-review 是后续真实评论来源或真实企微扩展的预留能力。当前前端不调用真实企微接口，企业微信演示全部在前端 mock 完成。

4. Prompt 设计与调优过程

后端按平台拆分两套评价生成 Prompt。

Google Prompt 目标：

- 输出英文评价。
- 使用北美真实消费者口吻。
- 内容客观、自然、具体，不像广告。
- 输出约 45-75 个英文词，最多约 420 个字符。
- 不提 AI、提示词、折扣或任务说明。

小红书 Prompt 目标：

- 输出中文推荐文案。
- 风格接近真实用户的小红书种草笔记。
- 适量 Emoji，分段有留白。
- 输出约 80-140 个中文字符，最多约220 个中文字符。
- 不提 AI、提示词、系统说明。

调优记录：

- 初始版本为了节省token，将 Google max_tokens 设置为 140，小红书设置为 180。
- 真实上线后出现前端没有正文的问题。排查发现不是数据库问题，也不是前端渲染问题，而是后端拿到的模型 message.content 为空。
- 后端增加空内容防护：如果模型第一次返回空内容，会用同样 token 上限重试一次；如果仍为空，则返回明确错误，不再让前端误判成功。
- 真实模型测试中，使用的模型在短 max_tokens 下仍可能返回空正文，因此最终把 Google 和小红书生成接口的 max_tokens 都提高到20000。
- Prompt 仍然限制最终输出字数，max_tokens=20000 只是给模型足够预算，避免推理模型或兼容代理把预算耗尽后正文为空。

相关代码：

```
Prompt 与 token 上限： backend/app/core/domain.py  
生成服务与空内容重试： backend/app/services/review_service.py  
模型 HTTP 客户端： backend/app/integrations/llm.py
```

5. Webhook 实现说明

任务原计划包含真实企业微信群机器人 Webhook。实际交付中，用户确认“当前情况下不接入真实企业微信，改为通过纯前端代码 mock 演示，重点展示 webhook 的数据拼凑与调用逻辑”。

当前 Webhook mock 流程：

1. 用户点击 “复制并模拟 Webhook” 。
2. 前端复制用户最终正文。
3. 前端根据平台、感受标签和正文生成中文摘要。
4. 前端生成商家回复草稿。
5. 前端拼装企业微信群机器人 Markdown JSON body。
6. 页面展示 mock 请求地址、请求方式、摘要、回复草稿和请求体。

mock 请求地址：

```
https://qyapi.weixin.qq.com/cgi-bin/webhook/send?key=MOCK_DEMO_KEY
```

说明：该地址只用于演示，不会真实发送企业微信群消息，也不需要配置 WECOM_WEBHOOK_URL。

6. API Key 安全与成本控制

模型 API Key 只放在后端环境变量中，不进入前端代码，不进入构建产物。

后端环境变量：

```
LLM_API_KEY  
LLM_BASE_URL  
LLM_MODEL  
LLM_TIMEOUT_SECONDS  
DATABASE_URL  
API_RATE_LIMIT_PER_MINUTE  
LLM_DAILY_REQUEST_WARNING_LIMIT  
API_USAGE_LOG_PATH  
FRONTEND_ORIGINS
```

成本控制措施：

- 用户每次流程只允许成功生成一次，生成后只能手动修改。
- 感受标签限制为 1-2 个，减少 Prompt 发散。
- Prompt 严格要求短文本输出，避免模型主动输出长文。
- 后端设置 LLM_TIMEOUT_SECONDS，避免模型请求无限等待。
- 后端加入 Demo 级内存限流，默认同一客户端每分钟最多 12 次模型相关请求。
- 后端写入模型调用审计日志，默认路径为 backend/logs/api-usage.jsonl，不记录 API Key，不记录评价正文。
- 每日模型请求量达到阈值后写入成本告警日志。

重要说明：当前生成接口的 max_tokens 已提高到20000，这是为了解决真实模型返回空正文的问题。由于 Prompt 仍要求短文本，正常输出不会变成长文；但公开 Demo 仍有成本风险，正式长期开放前建议接入更强的网关限流、验证码或访问码。

7. 数据库说明

本地 Docker PostgreSQL 已预留数据库和表：

```
容器名：backend-postgres-1
数据库：culture_media
用户：postgres
端口：127.0.0.1:5432
表：culture_media.public.reviews
```

reviews 表保存字段包括：

- 平台
- 外部评论 ID
- 顾客昵称
- 评分
- 感受标签
- 评论正文
- 中文摘要
- 商家回复草稿
- 企业微信发送状态
- 创建时间

当前主流程“生成评价 -> 复制 -> mock Webhook -> 跳转平台”不依赖数据库。Render 上如果没有可用 PostgreSQL，后端启动不会被数据库初始化阻塞；数据库只用于后续真实评论来源和入库扩展。

8. 部署说明

8.1 后端 Render

Render 服务：

```
服务名：cultrue-media
服务类型：Web Service
仓库：*****
分支：main
Root Directory：backend
Build Command：pip install -r requirements.txt
Start Command：uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

线上环境变量重点：

```
LLM_API_KEY=真实模型 Key
LLM_BASE_URL=真实模型 Base URL
```

```
LLM_MODEL=真实模型名称  
FRONTEND_ORIGINS=https://28344e2d.pinme.dev
```

已处理的问题：

- Render 首次部署时尝试连接本地 127.0.0.1:5432 PostgreSQL，导致启动失败；已改为数据库 schema 初始化失败只写 warning，不阻塞后端启动。
- PinMe 实际访问域名是 https://28344e2d.pinme.dev。

8.2 前端 PinMe

前端生产环境：

```
frontend/.env.production  
VITE_API_BASE_URL=https://cultrue-media.onrender.com
```

构建命令：

```
cd "C:\Users\36183\Desktop\boss\culture media\frontend"  
pnpm build
```

上传命令：

```
cd "C:\Users\36183\Desktop\boss\culture media"  
pinme upload frontend\dist
```

最终前端地址：

```
https://28344e2d.pinme.dev
```

9. 测试与验证记录

后端测试命令：

```
cd "C:\Users\36183\Desktop\boss\culture media"  
.\backend\.venv\Scripts\python.exe -X utf8 -m unittest discover -s backend\tests -t backend -v
```

最近验证结果：

```
22 个测试通过  
1 个 PostgreSQL 集成测试按配置跳过
```

前端构建命令：

```
cd "C:\Users\36183\Desktop\boss\culture media\frontend"  
pnpm build
```

已完成验证：

- GET https://cultrue-media.onrender.com/api/health 返回成功。
- CORS 预检已允许 https://28344e2d.pinme.dev 调用后端 POST /api/generate-review。
- 真实模型生成已跑通，之前空正文问题通过提高 max_tokens 到 20000 解决。
- 前端线上页面可打开，移动端布局可用。
- 生成成功后按钮锁定，只能手动修改。
- 从平台入口后退回页面后，仍可再次点击“打开发布入口”和“复制并模拟 Webhook”。

10. 未完成与后续扩展

当前任务书要求的 Demo 主流程已经完成并上线。剩余内容属于后续扩展或正式

类型	当前状态	后续处理
真实企业微信机器人	本次不接入真实机器人，只做前端 mock	如果后续要真实推送，需要重新启用后端 WECOM_WEBHOOK_URL 并调用 /api/notify-wecom
真实评论来源	已预留 /api/incoming-review 和 reviews 表，没有接 Google Business Profile API 或小红书真实来源	拿到平台授权或第三方评论来源后，把新评论转发到 /api/incoming-review
数据库线上化	本地 Docker PostgreSQL 已可用；Render 主流程不依赖数据库	如果要保存线上评论记录，需要配置 Render PostgreSQL 或其他公网 PostgreSQL
成本防护	已有 Demo 级限流和调用日志	长期公开前建议加验证码、访问码、网关级限流和真实成本告警
自动化前端测试	已做人工浏览器验证和构建验证	如长期维护，可补 Playwright/Cypress 自动化用例
PDF 交付	当前输出 Markdown 文档	本次已额外导出 PDF

11. 实际耗时

本项目从需求阅读、方案制定、前后端开发、真实模型调试、Render/Pin Me 部署、CORS 修复、token 空返回问题排查到最终文档整理，累计约 4-5 小时。主要耗时集中在真实部署、模型返回空正文排查和跨域配置修复。